Spring Cloud Netflix Ribbon

主要内容

Ribbon 简介
使用 Ribbon 开发微服务
集中式与进程内负载均衡区别
Ribbon 常见的负载均衡策略
Ribbon 的点对点直连

学习目标

知识点	要求
Ribbon 简介	了解
使用 Ribbon 开发微服务	掌握
集中式与进程内负载均衡区别	掌握
Ribbon 常见的负载均衡策略	掌握
Ribbon 的点对点直连	掌握

Spring Cloud Netflix Ribbon

一、 Ribbon 简介

Ribbon 是一个基于 Http 和 TCP 的*客服端*负载均衡工具，它是基于 Netflix Ribbon 实现的。它不像 spring cloud 服务注册中心、配置中心、API 网关那样独立部署，但是它几乎存在于每个 spring cloud 微服务中。包括 feign 提供的声明式服务调用也是基于该 Ribbon 实现的。ribbon 默认提供很多种负载均衡算法，例如 轮询、随机 等等。甚至包含自定义的负载均衡算法。Ribbon 可以用于解决并提供微服务的负载均衡的问题。

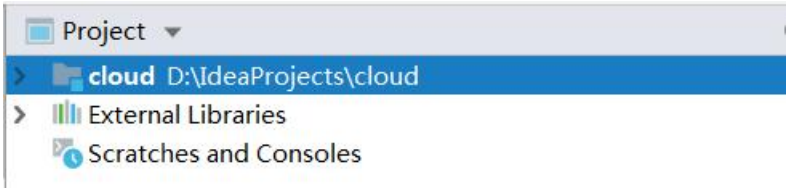
二、 使用 Ribbon 开发微服务

在 Spring Cloud 中，使用 Ribbon 技术开发 Eureka Client 组件还是非常方便的。我们在开发过程中，不需要像 Dubbo 那样关注服务的角色。无论是 Provider 还是 Consumer 都是一个微服务客户端，只是在编码层面上，服务消费者代码的开发相对比较麻烦。我们通过简单案例测试一下 Spring Cloud 中的微服务开发过程。

因为现在的 Eureka Server 部署在 Linux 中，并已为 Linux 定义了新的主机域名，需要先修改开发测试环境中的 hosts 文件。windows 中的 hosts 文件位于：C:\windows\system32\dirvers\etc\hosts。新增内容如下：（***IP 根据具体情况配置***）

```
192.168.2.115 eureka1
192.168.2.116 eureka2
```

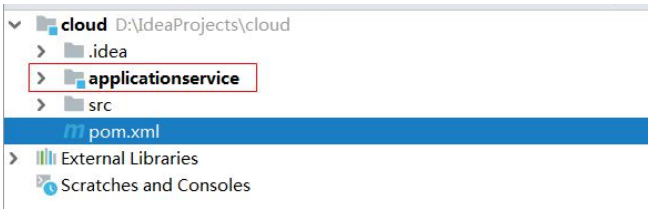
1 创建 Eureka Client 工程



1.1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <packaging>pom</packaging>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.2.2.RELEASE</version>
    </parent>
    <groupId>com.bjsxt</groupId>
    <artifactId>cloud</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>Hoxton.SR1</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>
</project>
```

2 创建 Application Service 服务提供者 Module



3 开发服务提供者 - Application Service

3.1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloud</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>applicationservice</artifactId>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
    </dependencies>

</project>
```

3.2 服务提供者代码

```
package com.bjsxt.controller;
```

```java
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ServiceController {
    @GetMapping
    public Object first(){
        return "测试Spring Cloud Netflix Ribbon开发服务提供者";
    }
}
```

3.3 配置文件 application.yml

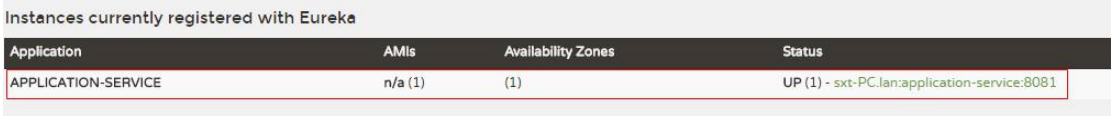
```yaml
server:
  port: 8081

spring:
  application:
    name: application-service

eureka:
  client:
    service-url:
      # 配置注册中心地址。如果注册中心为多个，则配置多行地址。
      # 注册中心地址格式为： http://ip:port/eureka/
      defaultZone:
        - http://eureka1:8761/eureka/
        - http://eureka2:8761/eureka/
```

3.4 启动类

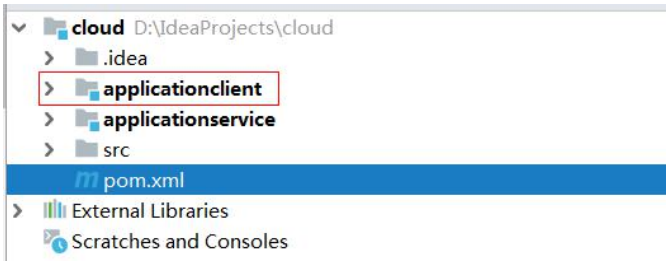
```java
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;

/**
 * @EnableEurekaClient注解 - 在Spring Cloud低版本开发中，每个Eureka Client
应用启动类上都必须增加此注解。
 *  代表当前应用是一个Eureka Client应用，即需要通过Eureka Server注册或发现服
```

```
务。
 */
@SpringBootApplication
public class ServiceApp {
    public static void main(String[] args) {
        SpringApplication.run(ServiceApp.class, args);
    }
}
```

3.5 检查 Eureka Server 中的服务注册情况



4 创建 Application Client 服务消费者 Module



5 开发服务消费者 - Application Client

5.1 POM 依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>cloud</artifactId>
        <groupId>com.bjsxt</groupId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>applicationclient</artifactId>
```

```
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
    </dependencies>

</project>
```

5.2 服务消费者代码

5.2.1 服务接口

```
package com.bjsxt.service;

public interface ClientService {
    String first();
}
```

5.2.2 服务实现

```
package com.bjsxt.service.impl;

import com.bjsxt.service.ClientService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service
public class ClientServiceImpl implements ClientService {
    @Autowired
    private LoadBalancerClient loadBalancerClient;
```

```
    /**
     * ribbon 负载均衡器，其中记录了从 Eureka Server 中获取的所有服务信息。
     */
    @Override
    public String first() {
        /*
         * ServiceInstance 封装了服务的基本信息，如 IP，端口
         * 在 Spring Cloud 中，对所有注册到 Eureka Server 中的服务都称为一个
service instance 服务实例。
         * 一个服务实例，就是一个有效的，可用的，服务单体实例或集群实例。
         * 每个 service instance 都和 spring application name 对应。
         * 可以通过 spring application name 查询 service instance
         */
        ServiceInstance si =
                this.loadBalancerClient.choose("application-service");
        // 拼接访问服务的 URL  http://ip:port/
        StringBuilder sb = new StringBuilder();
        sb.append("http://").append(si.getHost())
                .append(":").append(si.getPort()).append("/");

        System.out.println("本次访问的 service 是：  " + sb.toString());

        RestTemplate rt = new RestTemplate();

        ParameterizedTypeReference<String> type =
                new ParameterizedTypeReference<String>() {
                };

        ResponseEntity<String> response =
                rt.exchange(sb.toString(), HttpMethod.GET, null, type);
        String result = response.getBody();

        return result;
    }
}
```

5.2.3 控制器

```
package com.bjsxt.controller;

import com.bjsxt.service.ClientService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
```

```java
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ClientController {
    @Autowired
    private ClientService clientService;
    @GetMapping
    public Object first(){
        return this.clientService.first();
    }
}
```

5.3 配置文件 application.yml

```yaml
server:
  port: 8082

spring:
  application:
    name: application-client

eureka:
  client:
    service-url:
      defaultZone:
        - http://eureka1:8761/eureka/
        - http://eureka2:8761/eureka/
```

5.4 启动类

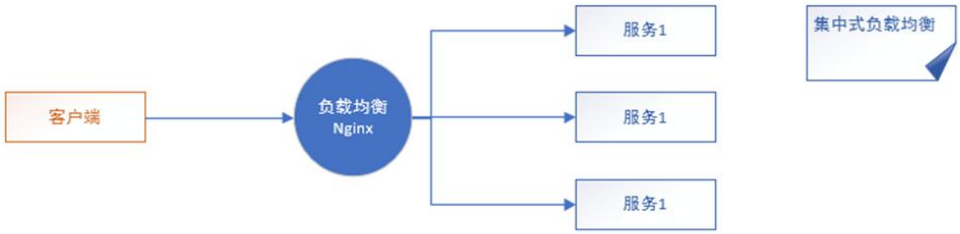
```java
package com.bjsxt;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ClientApp {
    public static void main(String[] args) {
        SpringApplication.run(ClientApp.class, args);
    }
}
```

三、 集中式与进程内负载均衡区别

业界主流的负载均衡解决方案有：集中式负载均衡和进程内负载均衡。

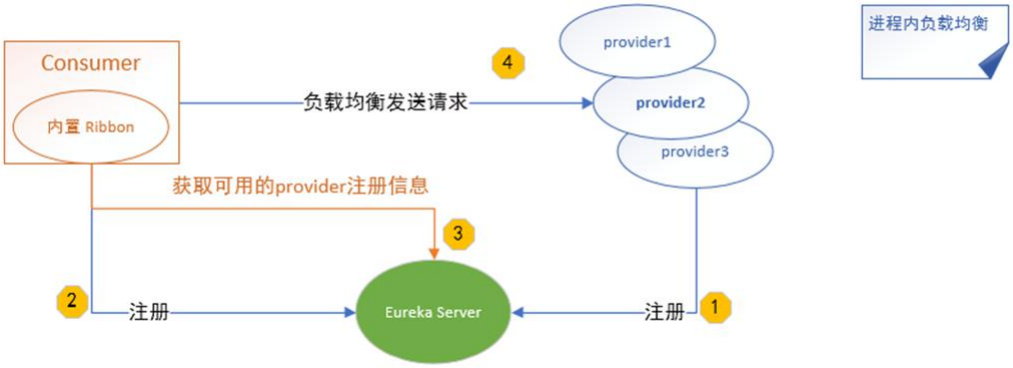
1 集中式负载均衡

即在客户端和服务端之间使用独立的负载均衡设施(可以是硬件，如 F5，也可以是软件，如 nginx)，由该设施负责把访问请求通过某种策略转发至服务端。



2 进程内负载均衡

将负载均衡逻辑集成到客户端组件中，客户端组件从服务注册中心获知有哪些地址可用，然后自己再从这些地址中选择出一个合适的服务端发起请求。Ribbon 就是一个进程内的负载均衡实现。



四、 Ribbon 常见的负载均衡策略

Ribbon 就属于进程内负载均衡，它只是一个类库，集成于 Eureka Client 进程，Eureka Client 进程通过访问注册中心 Eureka Server 发现服务列表，发现的服务列表信息是由 ribbon 来管理的。当访问 Application Service 的时候，Application Client 会通过 ribbon 来找到合适的 Application Service 地址信息，并发起远程调用请求。

1 Ribbon 中的常用负载均衡简介

ribbon 的负载均衡策略是通过不同的类型来实现的，下表详细介绍一些常用负载均衡

策略及对应的 Ribbon 策略类。

id	策略名称	策略对应的类名	实现原理
1	轮询策略（***默认***）	RoundRobinRule	轮询策略表示每次都顺序取下一个 provider，比如一共有 5 个 provider，第 1 次取第 1 个，第 2 次取第 2 个，第 3 次取第 3 个，以此类推
2	权重轮询策略（***常用***）	WeightedResponseTimeRule	1. 根据每个 provider 的响应时间分配一个权重，响应时间越长，权重越小，被选中的可能性越低。 2.原理：一开始为轮询策略，并开启一个计时器，每 30 秒收集一次每个 provider 的平均响应时间，当信息足够时，给每个 provider 附上一个权重，并按权重随机选择 provider，高权越重的 provider 会被高概率选中。
3	随机策略（***不推荐***）	RandomRule	从 provider 列表中随机选择一个 provider
4	最少并发数策略（***应用在硬件软件环境一致的情况下***）	BestAvailableRule	选择正在请求中的并发数最小的 provider，除非这个 provider 在熔断中。
5	在“选定的负载均衡策略”基础上进行重试机制	RetryRule	1. “选定的负载均衡策略”这个策略是轮询策略 RoundRobinRule 2.该重试策略先设定一个阈值时间

			段，如果在这个阈值时间段内当选择 provider 不成功，则一直尝试采用“选定的负载均衡策略：轮询策略”最后选择一个可用的 provider
6	可用性敏感策略（一般在同区域内服务集群环境中使用）	AvailabilityFilteringRule	过滤性能差的 provider, 有 2 种： 第一种：过滤掉在 eureka 中处于一直连接失败 provider 第二种：过滤掉高并发的 provider
7	区域敏感性策略(应用在大型的，物理隔离分布式环境中）	ZoneAvoidanceRule	1.以一个区域为单位考察可用性，对于不可用的区域整个丢弃，从剩下区域中选可用的 provider 2.如果这个 ip 区域内有一个或多个实例不可达或响应变慢，都会降低该 ip 区域内其他 ip 被选中的权重。

2 配置负载均衡策略

可以通过修改 Application Client 应用的全局配置文件来改变当前环境中使用的 Ribbon 负载均衡策略。

```
server:
  port: 8082

spring:
  application:
    name: application-client

eureka:
  client:
    service-url:
      defaultZone:
```

```
        - http://eureka1:8761/eureka/
        - http://eureka2:8761/eureka/

# 设置负载均衡策略。application-service 为设置负载均衡的服务名称
application-service:
  ribbon:
    NFLoadBalancerRuleClassName: com.netflix.loadbalancer.RandomRule
```

五、 Ribbon 的点对点直连

Ribbon 也可以脱离 Eureka Server 注册中心，通过配置的方式指定要调用的远程服务信息，实现 Ribbon 点对点直连。修改的配置内容如下：

```
server:
  port: 8082

spring:
  application:
    name: application-client

# application-service ：  要点对点直连的远程服务命名
application-service:
  ribbon:
    # 远程服务地址
    listOfServers: localhost:8081

# 关闭 ribbon 自动访问 eureka server 的功能。
ribbon:
  eureka:
    enabled: false
```